

Optimizing Cloud Data Lake Queries With a Balanced Coverage Plan

Grisha Weintraub , Ehud Gudes , Shlomi Dolev , *Fellow, IEEE*, and Jeffrey D. Ullman 

Abstract—Cloud data lakes emerge as an inexpensive solution for storing very large amounts of data. The main idea is the separation of compute and storage layers. Thus, cheap cloud storage is used for storing the data, while compute engines are used for running analytics on this data in “on-demand” mode. However, to perform any computation on the data in this architecture, the data should be moved from the storage layer to the compute layer over the network for each calculation. Obviously, that hurts calculation performance and requires huge network bandwidth. In this paper, we study different approaches to improve query performance in a data lake architecture. We define an optimization problem that can provably speed up data lake queries. We prove that the problem is NP-hard and suggest heuristic approaches. Then, we demonstrate through the experiments that our approach is feasible and efficient (up to $\times 30$ query execution time improvement based on the TPC-H benchmark).

Index Terms—Cloud storage, data lakes, query optimization.

I. INTRODUCTION

TRADITIONALLY, storage systems have been favoring data locality (meaning they wanted to be as close to the data as possible to speed up calculations on the data). In single-node databases, data locality occurs trivially, whereas in shared-nothing distributed systems (e.g., [1], [2], [3], [4]), data locality is achieved by performing computation on the same machines that store the data.

However, with the rise of cloud technologies, a new family of storage systems has emerged – cloud object stores (e.g., AWS S3 [5], Google Cloud Storage [6], and Azure Blob Storage [7]). These systems provide object storage service [8] through a web interface. Users create buckets, and each bucket may contain multiple binary objects uniquely identified within the bucket by a string key. Supported operations on objects in the bucket follow a simple key-value API:

Manuscript received 19 May 2023; revised 9 October 2023; accepted 1 December 2023. Date of publication 5 December 2023; date of current version 8 March 2024. This work was supported in part by the Israeli Council for Higher Education (CHE) via the Data Science Research Center, in part by the Israel Data Science Initiative (IDSI), and in part by Rita Altura trust chair in computer science and Israeli Science Foundation under Grant 465/22. Recommended for acceptance by S. Khan. (*Corresponding author: Grisha Weintraub.*)

Grisha Weintraub, Ehud Gudes, and Shlomi Dolev are with the Computer Science Department, Ben-Gurion University of the Negev, Beer-Sheva 84105, Israel (e-mail: grisha.weintraub@gmail.com; ehud@cs.bgu.ac.il; dolev@cs.bgu.ac.il).

Jeffrey D. Ullman is with the Stanford University, Stanford, CA 94305 USA (e-mail: ullman@gmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TCC.2023.3339208>, provided by the authors.

Digital Object Identifier 10.1109/TCC.2023.3339208

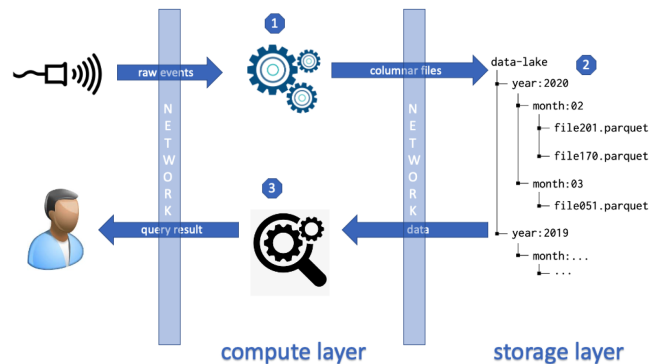


Fig. 1. Data lake system model.

- $get(k)$ – get the object with the key k
- $put(k, o)$ – put object o with associated key k into a bucket

It is common to construct object keys in a “file system” style. For example, we can simulate folder hierarchy by creating objects with keys “root/folder1/obj1” and “root/folder1/obj2”. Object stores support metadata “List-Objects” operation that allows users to get the keys of all objects from the particular “folder” (e.g., list on “root/folder1/” prefix would return “obj1” and “obj2”).

Cloud object stores are often recognized for their cost-effectiveness [9], [10], [11], [12]. As a result, they are heavily used as the main building block of enterprise data repositories that have come to be known as *cloud data lakes* [4], [9], [10], [11], [12], [13].

The main feature of the cloud data lakes is that they store data in cloud object stores and, as a result, do not follow the traditional shared-nothing architecture but, instead, disaggregate the compute layer from the storage layer. A typical approach for storing enterprise relational data in a cloud data lake looks as in Fig. 1:

- 1) Raw events arriving from the outside of the organization (e.g., from sensors) are processed by the dedicated ETL job (e.g., Apache Spark [14] or MapReduce [15]) and uploaded to the object store in a columnar format (e.g., Parquet [16] or ORC [17]). The columnar format provides efficient column-level compression and speeds up projection queries. Other formats, like JSON and CSV, are also supported.
- 2) Data lake files are usually organized into partitions [18] according to the domain business logic, which allows

skipping irrelevant files during the reads (e.g., “year” and “month” in Fig. 1).

- 3) Query engines (e.g., SparkSQL [19], Presto [20], Hive [18]) running on dedicated clusters execute SQL queries over the data in the lake.

Cloud data lakes introduce both benefits and challenges and, not surprisingly, are identified as a promising research direction by recent studies [10], [13]. Main benefits include:

- *Independent scaling of compute and storage layers:* For example, users can have very large data lakes (petabytes and beyond) but query them occasionally with an “on-demand” cluster. Such a scenario makes perfect sense for analytical use cases, but it is supported neither in traditional databases nor in Hadoop [3], where hardware is optimized for both storage and compute.
- *No vendor lock-in:* Disaggregated architecture and usage of standard tools in both storage and compute layers result in users’ flexibility in moving between different cloud providers. This also simplifies the adoption of new technologies, which in turn promotes research and innovation in this area.
- *General cloud benefits:* Cloud model brings a lot of advantages: pay-as-you-go, expert management, economies of scale resulting in a lower operational cost, and more [21].

The drawbacks are:

- *Poor query performance:* Data should be moved from the storage layer to the compute layer for each calculation, and that significantly hurts query performance. This issue is particularly problematic in interactive queries, as users have to wait an excessive amount of time for their results.
- *Difficulties with handing updates:* Object stores support only simple “put” operations, and as a result, there is no atomicity across multiple objects. In addition, mutations inside objects are not supported, so when there is a need to change the content of some file, the entire file should be rewritten.
- *Security issues:* There are many security-related challenges in the cloud model, including privacy, integrity, and availability issues [21].

In this paper, we focus on the first drawback (poor query performance); we discuss and define it formally in the following section.

Our main contributions are as follows:

- We identify the key challenges to improving query performance in cloud data lakes and discuss why existing solutions are not sufficient.
- We provide a theoretical model that formally defines the problem of poor query performance in cloud data lakes. The main component of the model is an optimization problem that clearly defines the relevant trade-offs. We prove that the problem is NP-hard and provide efficient ways to overcome its hardness.
- Based on our theoretical model, we design a generic “optimization framework” that provides a practical solution to the problem. The framework consists of three independent modules and can be implemented in different ways; we

TABLE I
SAMPLE METRIC DATA

	date	metric	val	...
1	2020-02-10	cpu	47	...
2	2020-02-14	cpu	58	... file201
3	2020-02-18	memory	11	...
4	2020-02-16	memory	8	...
5	2020-02-20	cpu	88	... file170
6	2020-02-21	cpu	66	...
7	2020-03-13	memory	6	...
8	2020-03-22	cpu	92	... file051
9	2020-03-28	cpu	71	...
...	...	...	...	...

```
select *
from metrics_table
where metric = 'memory' and val > 10
```

Fig. 2. Example query 1.

present our implementation and conduct large-scale experiments to show the efficiency of our approach.

The rest of the paper is structured as follows:

- In Section II we formally define the problem.
- In Section III we review related work.
- In Section IV we present our approach to the presented problem.
- In Section V we present our prototype implementation and experimental results.
- We conclude in Section VI.

II. PROBLEM STATEMENT AND PRELIMINARIES

Let us introduce the problem via a simple example first (we are going to use this example throughout the paper). Consider a typical metric data presented in Table I. Let us assume that this table is stored in the cloud data lake depicted in Fig. 1, such that records 1-3 are stored in “file201” records 4-6 are stored in “file170” and records 7-9 in “file051”. Files’ format can be any of the standard supported formats (e.g., Parquet, ORC, CSV, JSON, etc.).

Let us briefly review how the state-of-the-art query engines [18], [19], [20] would execute a typical query on Table I stored in the cloud data lake. For example, when a query engine receives Example Query 1 (Fig. 2), it reads the files from the storage (usually in parallel), scans them in-memory to find the records satisfying the predicate, and returns the result. In our example, only record 3 from *file201* will be returned. However, all the data lake files should be read from the storage and processed, and since production data lakes might contain billions of files [22], this approach is extremely wasteful.

Even though this example might look oversimplified, many real-world scenarios have the same limitation. In fact, any query that needs only a small part of the data lake for its calculation will suffer from the same problem. Examples of such scenarios include:

- Looking for a specific record(s) in the data lake for troubleshooting

TABLE II
NOTATION

notation	description
T	a data lake table
L	column names
D	domains
F	data lake files
Q	a data lake query
P_Q	a predicate in the “where” condition of query Q
$S(P_Q, t)$	tuple t satisfies predicate P of query Q
$Cov(X, Q)$	query Q is covered by files X
$TCov(X, Q)$	query Q is tightly covered by files X
$TC(Q)$	tight coverage set of Q
$TD(X, Q)$	tightness degree of the coverage set X of query Q
$CD(Q)$	coverage degree of the query Q
I_c	data lake index on column c
$ x $	size of object x
$C(x)$	cost of object x
$C_e(x)$	estimated cost of object x
K	upper limit of the balanced coverage plan cost
N	natural numbers $\{0, 1, 2, \dots\}$

- Fetching records related to a particular pattern for further analysis or ML training
- Managing the General Data Protection Regulation (GDPR) compliance [23], where records related to a specific user should be found in the data lake upon user request

So, intuitively, we would like to read only the “relevant” files for a given query (*file201* in our example) and skip all the rest. Let us define the problem formally now.

A. Formal Problem Definition

We model data lake tables according to the standard relational model [24]. Given a set of m domains $D = \{D_1, D_2, \dots, D_m\}$, a table (relation) T is defined as a subset of the Cartesian product $D_1 \times D_2 \times \dots \times D_m$. Each domain $D_i \in D$ has an associated column name $c_i \in L$, where L is the set of all column names. T is a set of tuples $\{t_1, t_2, \dots, t_{|T|}\}$, where each tuple t is a set of pairs $\{(c_i : v) \mid c_i \in L, v \in D_i, i \in \{1, \dots, m\}\}$.

T is stored in a cloud object store as a collection of files denoted by $F = \{f_1, f_2, \dots, f_{|F|}\}$. F is a partition of T , meaning that $\forall_{i \neq j} : f_i \subseteq T, \bigcup F = T, f_i \cap f_j = \emptyset$. For the purpose of the formal definition, we assume that all the files are of the same size, so all the read operations from the cloud are equivalent from the cost perspective.

Cost Model: In our cost model, we assume that the cost of query engine operations is dominated by the reads from the remote storage and denote the cost of operation x as $C(x)$ (i.e., to execute x , we need to perform $C(x)$ reads from the cloud storage). Our decision to use reads from the storage as the dominant component of the cost is consistent with the traditional DBMS approach [25], where the main cost metric is the number of I/O requests.

The notation used throughout the paper is presented in Table II.

Definition 1 (data lake query): We define a data lake query Q as a standard SQL query on table T and denote the predicate in the *where* clause of Q as P_Q . We assume that P_Q is given in a conjunctive normal form (CNF), which looks as follows: $(T_{11} \vee T_{12} \vee \dots) \wedge (T_{21} \vee T_{22} \vee \dots) \cdots \wedge (T_{n1} \vee T_{n2} \vee \dots)$. CNF is a

conjunction of *clauses* where a clause is a disjunction of *terms*. A term is a condition of type $\langle \text{column } \text{op} \text{ value} \rangle$ (e.g., $\text{age} > 40$) or $\langle \text{column1 } \text{op} \text{ column2} \rangle$ (e.g., $\text{salary} \leq \text{department-avg-salary}$), where columns are taken from L , values from domains in D , and $\text{op} \in \{=, \neq, \geq, \leq, >, <\}$.

If tuple t from the file $f \in F$ satisfies P_Q we denote it by $S(P_Q, t)$.

Definition 2 (query coverage): Given a data lake query Q , $X \subseteq F$ covers $Q \leftrightarrow \forall f \in F \setminus X, \neg \exists t \in f, S(P_Q, t)$.

When X satisfies Definition 2 for some data lake query Q , we say that X *covers* Q (meaning that X contains all the files needed to satisfy Q) and denote it by $Cov(X, Q)$. We call such X a *coverage set* of Q . Note that for any Q , holds $Cov(F, Q)$.

Definition 3 (query tight coverage): Given a data lake query Q , $X \subseteq F$ tightly covers $Q \leftrightarrow Cov(X, Q) \wedge \neg \exists f \in X, \forall t \in f, \neg S(P_Q, t)$

When X satisfies Definition 3 for some data lake query Q , we say that X *tightly covers* Q (meaning that X contains all the files needed to satisfy Q and *only* them) and denote it by $TCov(X, Q)$. We call such X a *tight coverage set* of Q and denote it by $TC(Q)$. By considering the example query above (Fig. 2), we can say that the query is covered by $\{\text{“file201” “file170” “file051”}\}$ but not by $\{\text{“file170” “file051”}\}$ and is *tightly covered* only by $\{\text{“file201”}\}$.

We can also extend the definitions of *coverage* and *tight coverage* to the record level (instead of the files level defined in Definitions 2, 3). In this case, tight coverage of the query Q in a record level is simply the actual record IDs returned by Q , and a coverage set of Q in a record level is any subset of table record IDs that contains the result record ids of Q .

If, for any data lake query Q , we could (efficiently) compute X such that $TCov(X, Q)$, we would be able to significantly improve query performance in a cloud data lake architecture by accessing only files in X instead of all those in F (and in most real-world scenarios $|X| \ll |F|$). In fact, as we show below, in many cases, finding the exact tight coverage might be too complicated, and we can be content with some coverage set that is not tight but still can help us improve query performance. For such scenarios, the definition of *tightness* and *coverage* degrees might be useful.

Definition 4 (tightness degree): Given a data lake query Q , for any coverage set X of Q , the tightness degree of X is defined as:

$$TD(X, Q) = \begin{cases} 1 - \frac{|X| - |TC(Q)|}{|F| - |TC(Q)|} & \text{if } |TC(Q)| < |F| \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Intuitively, the tightness degree shows to what extent the given coverage set is close to the tight coverage set (1 means a perfect match).

Definition 5 (coverage degree): Given the data lake query Q , the coverage degree of Q is defined as:

$$CD(Q) = \frac{|TC(Q)|}{|F|} \quad (2)$$

Coverage degree shows what part of the data lake must be scanned by Q . The best coverage degree is 0 (when no files need to be accessed), and the worst is 1 (when all files are required).

```

select *
from metrics_table
where year = '2020' and month = '02'

```

Fig. 3. Example query 2.

```

select *
from metrics_table
where val > 90

```

Fig. 4. Example query 3.

Based on the above semantics, we can formulate the main research questions of this paper as follows:

- 1) Can we develop an algorithm that, for any data lake query Q , can find a coverage set of Q , X , such that:
 - a) tightness degree of X is maximized
 - b) cost of computing X is minimized
 - c) as a result of the above, the total execution time of Q is reduced as much as possible
- 2) What are the practical considerations of applying the algorithm from question 1 in real-world systems?

Research question 1 above can be defined formally as the following multi-objective optimization problem [26]:

$$\begin{aligned}
& \text{minimize} \quad \{C(X), 1 - TD(X, Q)\} \\
& \text{subject to} \quad X \subseteq F, Cov(X, Q)
\end{aligned} \quad (3)$$

In the next section, we review related work with a focus on different approaches to the presented problem. In Sections IV and V, we present our approach to answering questions 1 and 2, respectively.

III. RELATED WORK

The most trivial approach for query execution in cloud data lakes is to read all the files. Clearly, F covers any Q , but the coverage is far from being *tight* (except specific cases where $CD(Q) = 1$), and hence query performance is poor.

A. Partitioning

One of the first suggested optimizations was data partitioning [18]. Consider again sample data in Table I where the table is partitioned by “year” and “month” columns. For queries whose predicates are based on partition columns, we can easily calculate the tight coverage set. For example, for the query in Example 2 (Fig. 3), we need to access only files in folder “year=2020/month=02” – {“file201” “file170”}.

Partitioning is a simple and powerful technique and is supported by all modern query engines (e.g., [4], [9], [14], [18], [20], [27]). Unfortunately, only a limited subset of table columns can be used in partitioning, while production tables may contain tens of thousands of columns [28]. As a result, global (cross-partition) queries cannot benefit from partitioning and need to scan all the data lake files. In our approach, on the other hand, we can skip irrelevant files for both global and local (partition-based) queries.

B. Data Skipping

Another well-known approach [29] is to attach metadata to each data lake file and use it during the reads to skip irrelevant files. For example, the query in Example 3 (Fig. 4) on Table I,

can use min/max values from the metadata section to calculate the query’s tight coverage set – “file051”.

Columnar formats support metadata-based skipping out-of-the-box [16], [17], [30] by storing the metadata and the data in the same file and relying on the fact that cloud object stores support reading of the particular sections of the file. To reduce the overhead of reading metadata from each file, recent studies suggest keeping all the metadata in a centralized place [9], [18], [28], [31] instead of per file.

Unfortunately, metadata-based skipping is very sensitive to data distribution and helps only in cases where the data is nicely clustered. For example, the query in Example 1 above (Fig. 2) cannot skip any of the files based on metadata, while its tight coverage set contains only a single file (“file201”). There have been several approaches to optimize data organization during the writes so it will maximize the benefits of metadata-based skipping [9], [32], [33], [34]; such approaches focus on improving specific queries and cannot serve as a general solution. In our approach, we do not rely on the data layout and can skip irrelevant files regardless of the way the data is distributed.

C. Predicate Pushdown

Some cloud providers in some cases (e.g., [11], [35], [36]) support pushing the query predicate to the storage layer, so irrelevant records might be filtered out during the read operation, and only the relevant records would be returned to the compute layer. This technique is a great optimization, as we do not need to move huge amounts of data between storage and compute layers. However, we still perform a lot of costly filtering operations; the only difference is that the operation is performed in a different place.

The predicate pushdown technique does not help with skipping the reading of irrelevant files but makes this operation faster. Thus, it cannot help with the problem we consider in this paper but can be combined with our technique as an optimization (we discuss it further in Section VI).

D. Data Warehouses

Data warehouses have been used for decades to optimize query performance and support business intelligence (BI) inside enterprises. In the cloud era, modern data warehouses (as Redshift [4], Snowflake [37], and BigQuery [38]) are used as a second tier in the data platform architecture: Data first is uploaded into the data lake (as described above) and later ingested into the data warehouse.

While certainly improving query performance, the data warehouse approach has the following significant drawbacks:

- *Maintenance complexity* - keeping two very large storage systems (data lake and warehouse) in sync is not a trivial

engineering task, which requires building and maintaining complex ETL processes. In addition, the data warehouse will always have stale data compared to that of the data lake, which can cause issues with BI flows.

- *Cost* - in addition to the ETL cost, the same data is stored twice, hence the double storage cost.

Ideally, we would like to find a solution that adds data warehouse capabilities to the data lake instead of keeping two different storage systems for the same data. This novel approach has been named *lakehouse* in recent studies [10], [22], [39], [40]. Our approach is a step towards achieving the “lakehouse” horizon.

E. Table Format

Table format (e.g., Delta Lake [9], Apache Iceberg [41], and Apache Hudi [42]) is a novel approach to add missing capabilities (e.g., transactions, schema evolution, query optimization) to the data lake architecture. The main idea is to add an additional layer of metadata between the files in the storage and the compute layer. This metadata can then be used, for example, for storing information about schema changes, data mutation, and various statistics to improve query performance.

While table format is an important step towards the lakehouse vision (and in some works, it is already considered to be a lakehouse [22]), it still does not solve the main problem considered in this paper: reading of (a lot of) irrelevant files from the storage. The main reason is the same as in data skipping (Section III-B) - if we have a predicate that is based on columns that are not clustered, we are unable to fully utilize file statistics and may end up reading unnecessary files; our approach tackles this problem effectively.

F. Caching

Most query engines use some sort of caching to improve query performance. For example, in [9], [37], data retrieved from the object store is cached on compute nodes’ local disks. In [19], users can explicitly cache query results in memory or on disk for subsequent usage.

Recent work of [11] introduced an interesting concept of “separable operators”. Simply put, for some query types (e.g., projections), we can get some of the data from the cache another part from the object store, and merge their results for the final output.

Caching clearly improves query performance, but unless we cache all the data lake on the compute layer, it does not prevent the reading of irrelevant files from the storage. However, we can combine our technique with caching to achieve better results. We discuss it further in Section VI.

G. Indexing

Indexing is the primary method for improving query performance in relational databases, and it sounds reasonable to apply these well-known techniques in the cloud data lakes as well. Just like traditional indexes map column values to their actual data blocks on disk, we can map data lake column values to their files

in the cloud object store, thereby finding a tight coverage set for the query.

However, whereas the general concept of using indexing in databases and data lakes is similar, some practical aspects are very different. For example:

- Big data volumes of cloud data lakes imply Big Data cloud indexes, and hence, highly scalable and cloud-native index implementation is required.
- Index in relational databases returns a list of record IDs that are mapped to the disk blocks, and in many cases, reading from disk via record IDs is less efficient than performing a full table scan (sequential read versus random read). The rule of thumb [25] says that “it is probably cheaper to simply scan the entire table if over 5% of the tuples are to be retrieved”. In the data lake model, on the other hand, sequential reads cannot cross multiple objects, and hence, it is always better to use an index result (even if the index returned 99% of the data lake files).

These differences introduce both opportunities and challenges. On the one hand, we cannot apply existing techniques as-is due to the different scales and environments. On the other hand, we can utilize the fact that having a tight coverage set is always beneficial to design much simpler and more efficient algorithms and data structures for indexing in cloud data lakes.

Existing indexing techniques for cloud data lakes include industry approaches like [43], where a simple inverted index is stored in a key-value store. This approach, however, does not consider a relational model and focuses on very simple queries on raw data. Our previous work in [44] presents an indexing scheme for relational data in cloud data lakes where indexes are stored inside the lake, and their creation is performed by parallel algorithms. This scheme, however, limits the query model to simple selection/projection queries with a single-column predicate.

To the best of our knowledge, there have not been deep studies on indexing in cloud data lakes based on the ideas from relational databases (neither in the industry nor in the academy). In the next section, we introduce our approach which is inspired by previous work on indexing in relational databases.

IV. OUR APPROACH

We start with an overview of our approach (Section IV-A) and then zoom into its building blocks (Section IV-B, IV-C, and IV-D).

A. Overview and Intuition

Let us first show what the problem is with the existing (naive) approach. Algorithm 1 presents the naive way of calculating the tight coverage set for a given query. As the algorithm reads all the data lake files from the cloud, its cost is $|F|$. Interestingly, that is also the best cost we can have in a general (worst-case) scenario, as assuming we can find the tight coverage set of an arbitrary query in sub-linear cost $o(F)$, would imply we can build a data structure of sub-linear size that can answer arbitrary search queries, and that would contradict the information-theoretic lower bound [45].

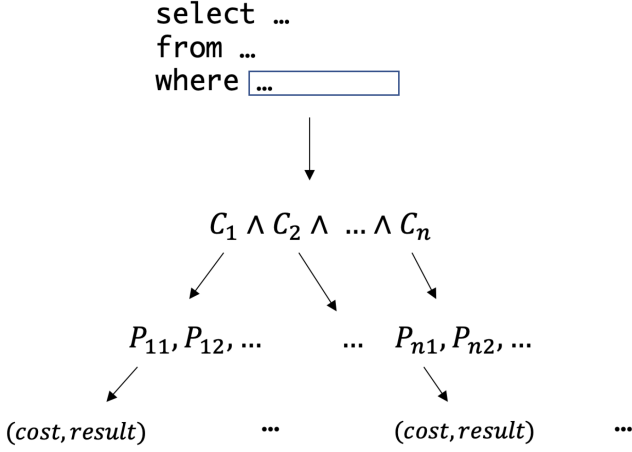


Fig. 5. Query estimations structure.

Algorithm 1: Naive Get Tight Coverage.**Input:** data lake query Q , data lake files F **Output:** $TC(Q)$

```

1:  $result \leftarrow \emptyset$ 
2: for each file  $f$  in  $F$  do
3:   read  $f$  from the cloud
4:   for each tuple  $t$  in  $f$  do
5:     if  $S(P_Q, t)$  then
6:        $result \leftarrow result \cup f$ 
7:       break
8:     end if
9:   end for
10: end for
11: return  $result$ 
  
```

Fortunately, for many practical scenarios, we can do better (in terms of the query performance cost) if we allow reducing the tightness degree. The main idea is, instead of looking for the *tight* coverage set of the given query, to find *some* coverage set that will result in an optimal *total* query execution time; we are looking for the coverage set X such that the sum of the cost of finding X and its size is minimized.

In our approach, for each data lake query, we focus on the “where” condition and look at each predicate clause separately (C values in Fig. 5). There may be many coverage sets associated with each clause, and there may be many ways to compute each of these coverage sets (P values in Fig. 5). We will assume that for each possible *coverage execution plan*, we can estimate (e.g., via statistics, caching, ML models, etc.) what is the expected cost and expected result of each plan (our approach for estimation is presented in Section IV-D). One example of a coverage plan is trivially to return F , which is a coverage set of any query. In this case, the cost of the plan is 0, as we do not read files from the cloud storage at all, and its result is F . Another example of a coverage plan is to run Algorithm 1. In this case, the cost is $|F|$, as we scan all the data lake files, and the result is $TC(Q)$, which can be any subset of F , depending on the query.

```

select *
from metrics_table
where
  (date = '2020-02-20' or date = '2020-03-13')
  and val < 7 and metric = 'cpu'
  
```

Fig. 6. Example query 4.

TABLE III
COVERAGE PLANS ESTIMATIONS FOR “EXAMPLE QUERY 4”

clause	est. cost	est. result
$C_1 = \text{"date='2020-02-20' or date='2020-03-13'"}$	5	{file170, file051}
$C_2 = \text{"val < 7"}$	1	{file051, file033, file302, file048}
$C_3 = \text{"metric='cpu'"}$	2	{file201, file170, file051, file079}

An important observation (defined formally in Theorem 1 below and proved in Appendix A, available online) is that the intersection of coverage sets of any subset of the query clauses is a coverage set of the original query. Based on this observation, we can explain the main idea of our approach (defined formally in Definition 6 below) as follows:

- 1) Given a query and its estimated values as in Fig. 5, we want to find a subset of clauses (and their corresponding coverage plans) such that the sum of their estimated costs and the size of the intersection of the coverage sets is minimized.
- 2) Then, we execute each of the coverage plans (in parallel), intersect their results and execute the original query on the files in the intersection only (which is a coverage set of the original query according to Theorem 1).

Theorem 1: Let Q be a data lake query, $C = \{C_1, C_2, \dots, C_n\}$ Q clauses, $X = \{X_1, X_2, \dots, X_n\}$ coverage sets such that $X_i \subseteq F$ is a coverage set of a data lake query whose “where” condition contains clause C_i only.

$$\forall X' \subseteq X, Cov \left(\bigcap_{x \in X'} x, Q \right) \quad (4)$$

Example 1: As an example, let us consider Example Query 4 (Fig. 6) and let us assume that its estimation values are given in Table III (we assume here a single plan per clause and that the results are estimated based on the hypothetical extension of Table I). Now, we are interested in finding a subset of $\{C_1, C_2, C_3\}$ that will provide us with the optimal total query execution performance. All the possible combinations are listed in Table IV along with their estimated total cost. We can see that the optimal cost(=4) is achieved by clauses $\{C_2, C_3\}$ with the coverage set {file051}.

To summarize this example, based on the given estimations in Table III, the optimal execution strategy for Example Query 4 is:

- 1) Get coverage set of C_2 (val<7) with the estimated 1 cloud read.

TABLE IV
COVERAGE PLANS COMBINATIONS FOR “EXAMPLE QUERY 4”

clauses	est. result	est. total cost
$\{C_1\}$	{file170, file051}	2 + 5 = 7
$\{C_2\}$	{file051, file033, file302, file048}	4 + 1 = 5
$\{C_3\}$	{file201, file170, file051, file079}	4 + 2 = 6
$\{C_1, C_2\}$	{file051}	1 + 5 + 1 = 7
$\{C_1, C_3\}$	{file170, file051}	2 + 5 + 2 = 9
$\{C_2, C_3\}$	{file051}	1 + 1 + 2 = 4
$\{C_1, C_2, C_3\}$	{file051}	1 + 5 + 1 + 2 = 9

- 2) Get coverage set of C_3 (metric='cpu') with the estimated 2 cloud reads.
- 3) Perform the query only on the files in the intersection of results from (1) and (2). The expected number of files is 1. So, in total, we perform around four reads from the cloud instead of $|F|$.

So, intuitively, we are looking for a balanced solution, where we balance between the estimated coverage plan cost and its result size. The following “Balanced Query Coverage Plan Problem” defines this problem formally.

Definition 6 (Balanced Query Coverage Plan Problem (BQCPP)): Given $C = \{C_1, C_2, \dots, C_n\}$ representing query clauses, with $C_i = \{P_{i1}, P_{i2}, \dots, P_{ik}\}$ being the set of possible coverage plans of the clause C_i , and $P_{ij} = (c, R)$, where $c \in N$ is the estimated cost of P_{ij} , and $R \in G$ is its estimated result. $(G, \otimes)$ is a semigroup where G represents a domain of possible plan results and binary operation $\otimes$ defines results' intersection. A function $size : G \rightarrow N$ assigns a positive integer value to each result in G . The problem is to find the smallest set of plans $X \subseteq P = \bigcup C$ such that at most one plan is taken for each clause and the sum of X plans' estimated costs and the $size()$ of their estimated results' intersection (by $\otimes$) is minimized (we call such X a balanced coverage plan):

$$X = \underset{X}{\operatorname{argmin}} \left\{ |X| : X \in \underset{Y \subseteq P}{\operatorname{argmin}} \left(\sum_{y \in Y} y.c + size \left(\bigotimes_{y \in Y} y.R \right) \right), \right. \\ \left. \neg \exists i, k, j : P_{ij} \in X, P_{ik} \in X \right\} \quad (5)$$

Definition 7 (Coverage Plan Estimated Cost): The estimated cost of the given coverage plan P is defined as:

$$C_e(P) = \left(\sum_{p \in P} p.c + size \left(\bigotimes_{p \in P} p.R \right) \right) \quad (6)$$

Looking again at Example 1 above, we can define BQCPP parameters for this example as follows:

- $C = \{C_1, C_2, C_3\}$
- $C_1 = \{(5, \{\text{"file170"}, \text{"file051"}\})\}$, $C_2 = \{(1, \{\text{"file033"}, \text{"file051"}, \text{"file048"}, \text{"file302"}\})\}$, $C_3 = \{(2, \{\text{"file170"}, \text{"file051"}, \text{"file201"}, \text{"file079"}\})\}$
- $G = 2^F$, $\otimes$ is set intersection, $size()$ is set cardinality

To achieve more accurate results, instead of estimating coverage results as files, we can estimate them at the records level. Then, G would be defined as 2^T , and $size$ would be defined as

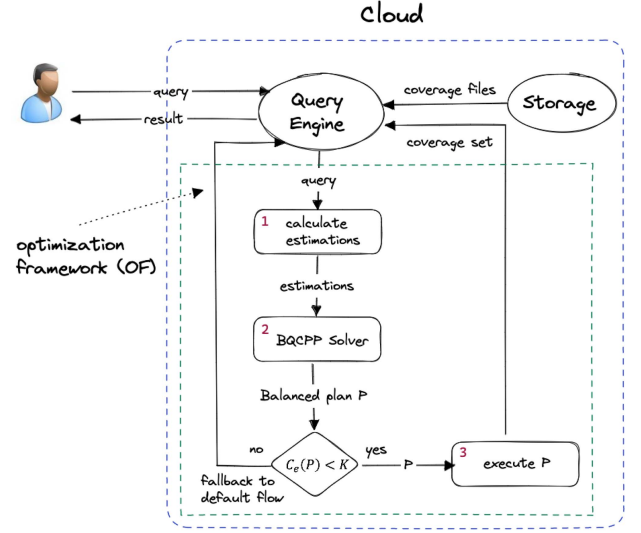


Fig. 7. High level architecture.

$size(R) = |\{f \mid \exists t \in R, t \in f, f \in F\}|$. Similarly, we can use BQCPP in many other different variations.

BQCPP clearly gives us what we need - the optimal plan to calculate a coverage set for the given query. However, it introduces two significant challenges: first, it is not clear how we can efficiently estimate coverage files or records for a given clause; second, according to Theorem 2, it is NP-hard (proved in Appendix B, available online). We deal with both these challenges in the following sections.

Theorem 2: BQCPP is NP-hard when $\otimes$ is defined as set intersection, and $size()$ is defined as set cardinality.

Now we can outline the high-level architecture of our overall solution (Fig. 7):

- 1) Client submits a query to the query engine.
- 2) Query engine passes the query to our “Optimization Framework” (OF), which is denoted by the green dotted square in the diagram. OF can be embedded in the query engine or run as a separate service.
- 3) OF consists of the following three modules (marked by red numbers in the diagram):
 - a) Estimations calculation - this module receives the original query from the query engine and prepares estimation values in the format of BQCPP input defined in Definition 6 (and depicted in Fig. 5).
 - b) BQCPP Solver - this module receives BQCPP input from module 1 and solves the BQCPP problem to find a balanced coverage plan P . We want to be sure that OF improves query performance and does not degrade it, so we check that the balanced coverage plan P has an estimated cost below the predefined threshold K (if it does not, we fallback to the default flow).
 - c) Coverage plans execution - module 3 gets the balanced coverage plan P from module 2 and executes it to get the corresponding coverage set (each one of the plans from P is executed in parallel, and

their results are intersected) and returns it to the query engine.

- 4) Query engine reads the coverage set files from the storage, executes the query based on the retrieved files, and returns the query result to the client.

Our architecture is based on the following two core principles:

- **Do no harm:** We want to be sure that we do not degrade existing query performance; we can either improve it or keep it as-is. In our optimization framework, modules 1 and 2 should have all the required information in-memory and hence, in our cost model, can not increase the cost of the query. The only component that can increase the cost is module 3, but it is *cost-bound*, meaning that users can limit the maximal estimated cost of the balanced coverage plan, and in case this threshold (K) is reached, fallback to the default (existing) strategy. An obvious threshold is $|F|$, but because the cost is estimated, different users can use different thresholds according to their datasets and needs.
- **Pluggability:** It can be seen that our architecture is generic and agnostic to the actual types of the different components. We can run on any cloud, use any query engine, storage layer, etc. New components introduced in our optimization framework (BQCPP algorithms, coverage plans, estimations) also can be replaced by more efficient (or appropriate) implementations.

In the following sections, we explain how we implement each one of the OF modules. First, we explain how we solve BQCPP (Section IV-B); then how we calculate coverage sets (Section IV-C); and finally, how we calculate estimation values (Section IV-D). It is important to note here that our implementation of the modules is only one of the many possible options. Our architecture allows replacing each of the modules with another implementation and even mixing different strategies in the same module. The only constraint is to respect the following basic contract:

- Module 1 gets a query and returns corresponding estimations as BQCPP input (Definition 6).
- Module 2 gets BQCPP input and returns BQCPP output (Definition 6).
- Module 3 runs the given coverage plan and returns the corresponding coverage set.

B. BQCPP Solver (Module 2)

BQCPP is NP-hard when $\otimes$ is defined as set intersection, and $size()$ is defined as set cardinality (proved in Appendix B, available online). To deal with its hardness, we propose the following two simple strategies:

- 1) **Optimistic** (Algorithm 2): We assume here that in most real-world scenarios, the number of clauses and plans is relatively small (at most dozen clauses per query with one or two plans per clause). In such scenarios, we can simply iterate through all the options and choose the best one (just like we did in the example presented in Table IV above). Since the number of options is fixed, the running time is $O(1)$.

Algorithm 2: Optimistic BQCPP Solver.

Input: clauses C with the associated plans and estimations, functions $\otimes$, $size()$

Output: balanced coverage plan

```

1: out  $\leftarrow \emptyset$ 
2: for each  $X \subseteq \bigcup C$  s.t.  $\neg \exists i, k, j :$ 
    $P_{ij} \in X, P_{ik} \in X$  do
3:   cost  $\leftarrow \sum_{x \in X} x.c$ 
4:   size  $\leftarrow size(\otimes_{x \in X} x.R)$ 
5:   sum  $\leftarrow cost + size$ 
6:   if out =  $\emptyset$  or sum <  $\sum_{x \in out} x.c + size$ 
        $(\otimes_{x \in out} x.R)$  then
7:     out  $\leftarrow X$ 
8:   end if
9: end for
10: return out

```

- 2) **Greedy** (Algorithm 3): For predicates with many clauses and/or plans, we can perform a simple greedy algorithm: Iterate over all the plans (lines 4–12), while considering at most one plan per clause, and on each iteration add a plan to the result set if it produces the minimal total cost (line 14). We stop when no plan can improve the cost (line 16) of the current set or the plans for all the clauses were added (line 18). In the worst case we iterate through all the clauses and for each clause through all the plans, so the complexity of the greedy algorithm is $O(m^2k)$ (where m is the number of clauses and k is the number of plans per clause). Considering the example estimations in Table III again, the greedy algorithm would start with the clause that has the minimal total cost (C_2), then it would add C_3 as it reduces the cost from 5 to 4, and then it would stop as adding C_1 would increase the cost from 4 to 9.

C. Coverage Sets Calculation (Module 3)

Here, we focus on module 3 from Fig. 7. We assume that we got from the previous module (2) a balanced coverage plan P , and we need to execute it to get the corresponding coverage set. Since we present a single strategy, we do not need the granularity of plans and can focus on clauses only. Let $C = \{C_i, C_j, \dots\}$ be clauses associated with the given coverage plan P . Our goal is to find the corresponding coverage set. Our approach is based on indexing. We first explain how we build and update our indexes (Section IV-C1) and then how we use them to calculate query coverage sets (Section IV-C2).

- 1) **Index Computation:** We start with the following definition of the *data lake index*.

Definition 8 (data lake index):

$$I_{c_i} = \{(v, j, k) \mid \exists t_k \in f_j, f_j \in F, (c_i, v) \in t_k, c_i \in L, v \in D_i\} \quad (7)$$

Less formally, we can say that the *data lake index* on column c in table T is a relation with the following three columns:

- *value* - all non-empty values of c in T (v in Definition 8)

Algorithm 3: Greedy BQCPP Solver.

Input: Clauses C with the associated plans and estimations, functions $\otimes$, $size()$

Output: approximated balanced coverage plan

```

1: out  $\leftarrow \emptyset$ 
2: repeat
3:    $T_1 \leftarrow \text{out}$ 
4:   for each  $P_{ij} \in \bigcup C$  s.t.  $\neg \exists P_{ik} \in \text{out}$  do
5:      $T_2 \leftarrow \text{out} \cup P_{ij}$ 
6:      $\text{cost} \leftarrow \sum_{x \in T_2} x.c$ 
7:      $\text{size} \leftarrow size(\otimes_{x \in T_2} x.R)$ 
8:      $\text{sum} \leftarrow \text{cost} + \text{size}$ 
9:     if  $T_1 = \emptyset$  or  $\text{sum} < \sum_{x \in T_1} x.c + size(\otimes_{x \in T_1} x.R)$ 
10:      then
11:         $T_1 \leftarrow T_2$ 
12:      end if
13:   end for
14:   if  $\sum_{x \in T_1} x.c + size(\otimes_{x \in T_1} x.R) < \sum_{x \in \text{out}} x.c + size(\otimes_{x \in \text{out}} x.R)$  then
15:     out  $\leftarrow T_1$ 
16:   else
17:     break
18:   end if
19: until  $|\text{out}| < |C|$ 
20: return out

```

- *file* - file id where a particular column value appears (j in Definition 8)
- *record*¹ - record id where a particular column value appears (k in Definition 8)

Using this definition, indexes built on columns “value” and “date” from Table I would look as in Table V. Our index maps column values to their records. This design allows us to calculate the tight coverage set for any given query; however, it requires relatively high storage overhead. For some scenarios (e.g., needle in a haystack [44]), it may be sufficient to use the index only at the file level, which would require much less storage.

We store index tables in the data lake as regular tables sorted by the indexed column value. To speed up index queries, we build the *root* index - a table that serves as an index of indexes. The root index stores per each index file a single record that summarizes the most important statistics about the file. The creation of the root index can be represented by the query in Fig. 9. An example of the root index for the indexes in Table V might look as in Table VI. The root index allows us to efficiently locate relevant index files by checking their min/max values. We assume that the root index is cached by the query engine.

Creation of index files, as well as the root index update, is performed by the same ETL that inserts new bulk of data into

¹For the sake of the formulation simplicity, we use global record id (in a table level) to define the “record-id” in Definition 8, but record id in a file-level would be sufficient as well

TABLE V
INDEX EXAMPLE

I_{val}			I_{date}		
value	file	record	value	file	record
6	051	7	2020-02-10	201	1
8	170	4	2020-02-14	201	2
11	201	3	2020-02-16	170	4
47	201	1	2020-02-18	201	3
58	201	2	2020-02-20	170	5
66	170	6	2020-02-21	170	6
71	051	9	2020-03-13	051	7
88	170	5	2020-03-22	051	8
92	051	8	2020-03-28	051	9

TABLE VI
ROOT INDEX EXAMPLE

col	file	min	max	cnt	cntd
val	index11	6	47	4	4
val	index12	58	92	5	5
date	index21	2020-02-10	2020-02-20	5	5
date	index22	2020-02-21	2020-03-28	4	4

```

select *
from metrics_table
where (date = '2020-02-20' or date = '2020-03-13')
and val > 80

```

Fig. 8. Example query 5.

the lake (step 1 in Fig. 1). Just like in relational databases, we create indexes only for searchable columns defined by a user and only those that have a relatively high cardinality.

2) *Index Usage:* Algorithm 4 presents a function that computes the tight coverage set of the query with the given set of clauses C , assuming the availability of data lake indexes (I_{c_i}) for each column appearing in the clauses. Note that we compute the tight coverage of the subset of the original query clauses and not the tight coverage of the original query (we rely here on Theorem 1), and hence we do not need to build indexes on all columns that appear in the “where” condition, only the most relevant ones. Algorithm 4 can be easily parallelized (our implementation based on Apache Spark [14] is available in [46]).

Algorithm 4 iterates over the clauses (line 2) and clauses’ terms (line 4) and for each term, computes a relation representing files and records satisfying the corresponding term (lines 6, 9). Files and records are computed based on relevant indexes. For terms of type $\langle \text{column} \text{op} \text{value} \rangle$, we simply apply the given operation and value on the corresponding index; for terms of type $\langle \text{column1} \text{op} \text{column2} \rangle$, we join between two corresponding indexes on columns “value” and “record” (value is joined by applying “op” while the record is joined by equality). The results are union-ed between terms in the same clause and intersected between different clauses (line 12).

Let us demonstrate how based on Algorithm 4 we can compute the tight coverage set of Example Query 5 from Fig. 8 (based on data from Table I):

- for the term “date = 2020-02-20”, $\text{temp}_1 = \{(170, 5)\}$
- for the term “date = 2020-03-13”, $\text{temp}_2 = \{(051, 7)\}$
- for the term “val > 80”, $\text{temp}_3 = \{(170, 5), (051, 8)\}$
- final result is $(\text{temp}_1 \cup \text{temp}_2) \cap \text{temp}_3 = \{(170, 5)\}$

Algorithm 4: Get Tight Coverage By Index.

Input: clauses C of Query Q , data lake indexes $I_{c_i}, i \in \{1, \dots, m\}$
Output: $TC(Q)$

```

1:  $result \leftarrow U$ 
2: for each clause  $c$  in  $C$  do
3:    $temp \leftarrow \emptyset$ 
4:   for each term  $t$  in  $c$  do
5:     if  $t$  is of type  $\langle c_i \text{ op } v \rangle$  then
6:        $temp \leftarrow temp \cup \pi_{file, record}(\sigma_{value \text{ op } v_i}(I_{c_i}))$ 
7:     end if
8:     if  $t$  is of type  $\langle c_i \text{ op } c_j \rangle$ 
9:        $temp \leftarrow temp \cup \pi_{file, record}(I_{c_i} \bowtie_{value, record} I_{c_j})$  then
10:    end if
11:  end for
12:   $result \leftarrow result \cap temp$ 
13: end for
14: return  $\pi_{file}(result)$ 

```

- hence, the tight coverage of Query 5 is $\{170\}$

D. Estimations (Module 1)

As an input to BQCPP Solver (module 2), we need to provide the estimated cost and estimated result of Algorithm 4 for each of the clauses in the given query. As mentioned above, we can ignore clauses containing non-indexed columns (Theorem 1), so we set the estimated cost for all such clauses as $+\infty$ and their estimated result as F . For the rest of the clauses, we perform the following calculation.

Cost Estimation: The cost of a particular clause $C_i = T_{i1} \vee T_{i2} \dots \vee T_{ik}$ can be estimated precisely by the following calculation based on the root index:

- 1) For terms of type $\langle col \text{ op } val \rangle$ we scan the root index to find ranges that can satisfy the term based on min/max values. The estimated cost of the term ($C_e(T)$) is the number of corresponding index files.
- 2) For terms of type $\langle col1 \text{ op } col2 \rangle$ we need to perform a full scan of indexes I_{col1}, I_{col2} , so the estimated cost of the term, in this case, is the number of index files in both indexes: $C_e(T) = |I_{col1}| + |I_{col2}|$.
- 3) Finally, the estimated cost of disjunction of two terms $T_1 \vee T_2$ is equal to $C_e(T_1) + C_e(T_2)$. Hence, the estimated cost of the particular clause is the sum of the estimated costs of its terms - $C_e(C_i) = C_e(T_{i1}) + C_e(T_{i2}) + \dots + C_e(T_{ik})$

Result Estimation: The result estimation is based on the regular relational databases *selection size estimation* [47], in which we can estimate the number of records satisfying the predicate based on the statistical information (i.e., G value in Definition 6 equals N). In our case, statistics are stored in the root index (which can serve as a histogram for the estimation needs). For example, the term of type “column = val” can be estimated by the query in Fig. 10.

Once we have the estimated number of records (R) for the given clause, we can estimate the corresponding number of

TABLE VII
BENCHMARK TABLES SIZES (GB)

num of files	CSV	Iceberg	Parquet
10,000	757	164	233
20,000	756	175	239
50,000	756	177	242
100,000	756	180	245

files ($size(R)$) as the number of non-empty bins after randomly throwing R balls into $|F|$ bins:

$$size(R) = |F| \cdot \left(1 - \left(\frac{|F| - 1}{|F|}\right)^R\right) \quad (8)$$

When the result is estimated by the number of records, the intersection ($\otimes$) is calculated as:

$$\otimes(R_1, R_2) = \left\lfloor \frac{R_1 \cdot R_2}{|T|} \right\rfloor \quad (9)$$

where $|T|$ is the total number of records in the table. In this case, BQCPP is not NP-hard and can be solved in polynomial time by Algorithm 3. The reason why the problem becomes simpler is that when using numbers for the results estimation and the equation above for intersection, each subset with the minimal cost must be included in the final result, and that drastically reduces the number of possible solutions.

V. EXPERIMENTS

We built a prototype of our solution; the implementation is available online [46]. We used Apache Spark (3.3.0) as the main compute engine for index creation and query evaluation. We used AWS as the cloud provider, its EMR platform (6.9.0) for running Spark jobs, and S3 as a storage service. For all the experiments, we used an EMR cluster with the same hardware configuration - 10 nodes of m5.2xlarge instance type (each with 8 vCore, 32 GB memory, 128 GB EBS storage).

For the benchmark, we used the TPC-H dataset (3.0.1) [48] with a scale factor of 1TB. We generated the largest (lineitem) table with around 6 billion records and stored it in S3 in three different formats (CSV, Iceberg, Parquet) and four different settings of a number of files (10 k, 20 k, 50 k, 100 k). In total, we got 12 instances of the same table (see Table VII for details).

We then created indexes for each of the 12 tables (according to the Definition 8) on three columns (extendedprice, shipdate, commitdate) and stored them in S3 as 3,000 Parquet files (1,000 files per each indexed column) with the average total size of 344GB. The root index was created as a single newline-delimited JSON file according to the query in Fig. 9 with an average size of 654KB. The root index was stored in S3 and cached in the Spark cluster during the query evaluation. Index and root index creation by the dedicated Spark job [46] took around 1 h per table.

As benchmark queries, we used TPC-H Query 1 (Fig. 11) and Query 6 (Fig. 12). We used different query arguments and additional conditions to simulate queries with various query coverage degrees. The complete details about each query used in the benchmark are available in [46].

```

create table root_index as
(
  select indexed_column_name, index_filename,
  max(value), min(value),
  count(*), count(distinct value)
  from index
  group by indexed_column_name, index_filename
)

```

Fig. 9. Root index.

```

select cnt/cntd
from root_index
where col = column and val >= min
and val <= max

```

Fig. 10. Records number estimation.

```

select
  l_returnflag,
  l_linestatus,
  sum(l_quantity) as sum_qty,
  sum(l_extendedprice) as sum_base_price,
  sum(l_extendedprice*(1-l_discount)) as sum_disc_price,
  sum(l_extendedprice*(1-l_discount)*(1+l_tax)) as sum_charge,
  avg(l_quantity) as avg_qty,
  avg(l_extendedprice) as avg_price,
  avg(l_discount) as avg_disc,
  count(*) as count_order
from
  lineitem
where
  l_shipdate <= date '1998-12-01' - interval 'DELTA' day (3)
group by
  l_returnflag,
  l_linestatus
order by
  l_returnflag,
  l_linestatus;

```

Fig. 11. TPC-H query 1 (taken from [48]).

```

select
  sum(l_extendedprice*l_discount) as revenue
from
  lineitem
where
  l_shipdate >= date '[DATE]'
  and l_shipdate < date '[DATE]' + interval '1' year
  and l_discount between [DISCOUNT] - 0.01 and [DISCOUNT] + 0.01
  and l_quantity < [QUANTITY];

```

Fig. 12. TPC-H query 6 (taken from [48]).

In our evaluation, we executed queries with and without our scheme. We verified that both executions returned exactly the same result. We set the upper limit of the balanced coverage plan cost K as the number of data lake files $|F|$ in all experiments. The main results of the evaluation are presented in Figs. 13, 14, 15, and 16 (full experimental details are available in [46]).

Graphs in Figs. 13, 14, 15, and 16 compare our scheme with the following three approaches by query run time:

- CSV - the baseline of the TPC-H benchmark without any optimizations discussed in Section III.

- Iceberg - table format/lakehouse approach (Section III-E)
- Parquet - data skipping approach (Section III-B)

To better understand the trade-offs, we added the following dimensions to our evaluation:

- TPC-H query type - Q1 (Fig. 11) or Q6 (Fig. 12)
- Query coverage degree - we have 14 variations of TPC-H queries, each with a different coverage degree; the higher the query number, the higher the coverage degree
- Number of files in the data lake - 10,000, 20,000, 50,000, 100,000

A. Results Analysis and Discussion

Before starting the analysis of the experiments, let us recall the main steps of our approach (Fig. 7):

- For each query, we first extract the predicate from the “where” condition and estimate the cost and result of the coverage plan for each of the predicate clauses. We get a data structure as in Fig. 5.
- Based on the estimated values from the previous step, we find the best (balanced) coverage plan for the query. We use Algorithm 3 in the benchmark.
- If the estimated cost of the balanced coverage plan is higher than the predefined threshold K , we fallback to the default flow. Otherwise, we execute the plan to get a balanced coverage set for the query. Technically, we are using indexes to find the coverage set for each of the clauses in the balanced plan and then intersect their results to get the final coverage set.
- Finally, we execute the query only on the files found in the balanced coverage set from the previous step.

So, from the practical perspective, we want to achieve the following two goals:

- 1) Improve query execution as much as possible (by running on a subset of data lake files).
- 2) Do not degrade the existing query execution as much as possible (by successfully estimating the cost of our approach and falling back to the default strategy if our execution is slower).

In the below analysis, we assess to what extent we succeeded in these two goals, what are the limitations of our approach and how we can overcome them.

1) Analysis of Query Run Time Improvement: We have four different dimensions in our evaluation - query type (Q1/Q6), query coverage (1–14), data format (CSV, Iceberg, Parquet), and number of files (10 k/20 k/50 k/100 k). Below, we discuss the impact of each of them on the query run time.

Query Type: We can see that in all graphs (Fig. 13, 14, 15, and 16), Q1 takes more time than Q6, and that is consistent across all other dimensions with and without our scheme. That result makes sense as Q6 (Fig. 12) is a simple table scan, while Q1 (Fig. 11) is a “group by” operation. Besides having different run times, Q1 and Q6 behave similarly in all other aspects, demonstrating that our approach is query-type agnostic.

Query Coverage: As expected, the existing approaches (without our scheme) are almost unaffected by the query number,

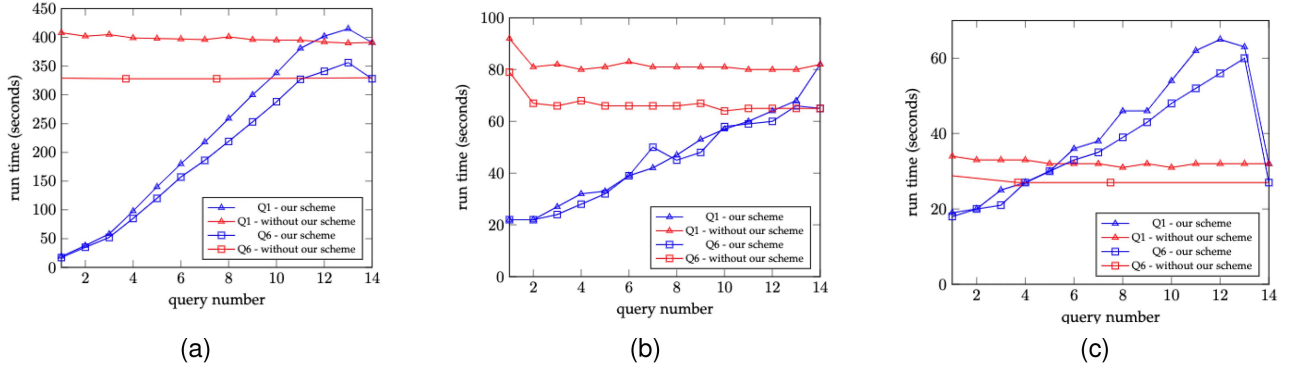


Fig. 13. Evaluation of TPC-H Q1 and Q6 on a table with 10,000 files. (a) CSV. (b) Iceberg. (c) Parquet.

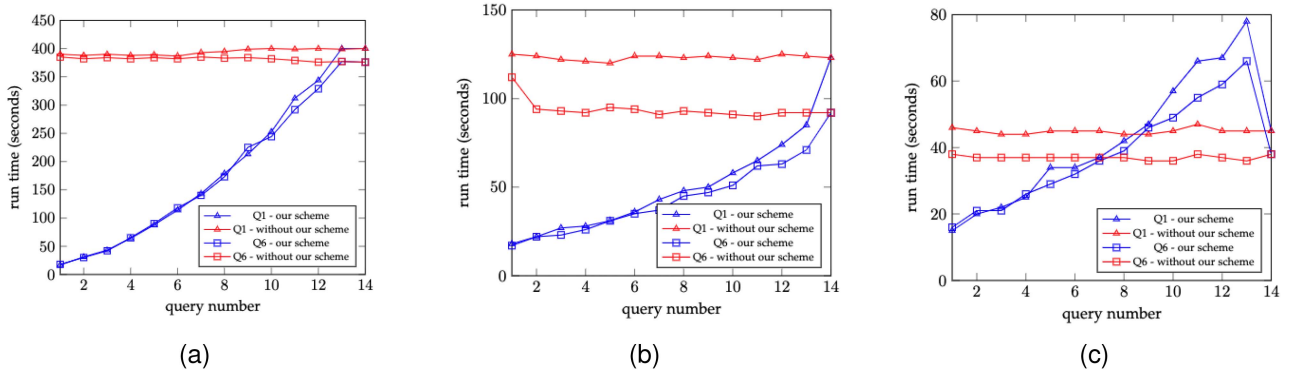


Fig. 14. Evaluation of TPC-H Q1 and Q6 on a table with 20,000 files. (a) CSV. (b) Iceberg. (c) Parquet.

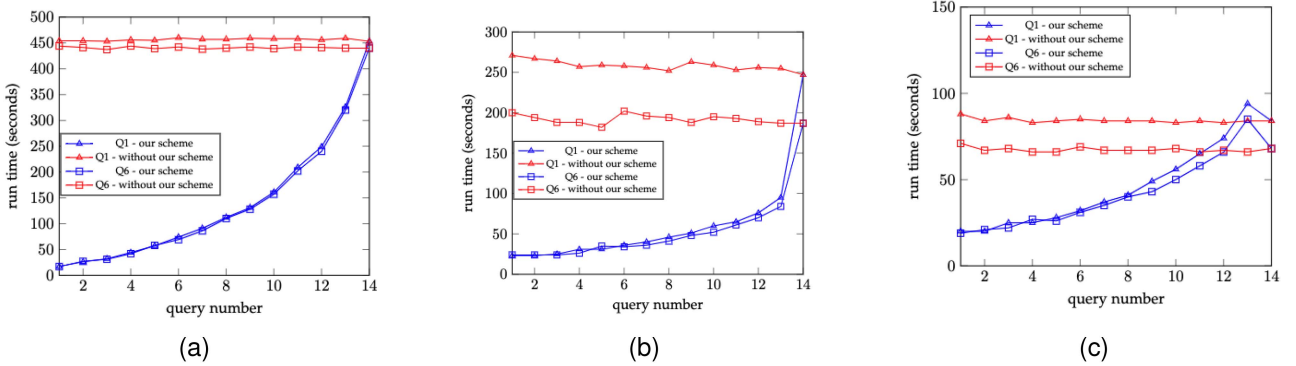


Fig. 15. Evaluation of TPC-H Q1 and Q6 on a table with 50,000 files. (a) CSV. (b) Iceberg. (c) Parquet.

as they need to access all the files regardless of the coverage degree. In our scheme, on the other hand, it can be clearly seen that we perform best for queries with a low coverage degree (up to $\times 30$ times faster as in Fig. 16(a)). This result is expected as our approach is based on the idea of reading only the relevant files from the data lake, and when the coverage degree is low, we need to read fewer files.

Number of Files: With the dimension of “number of files”, we have a similar pattern to “query coverage” but in the opposite direction. As the number of files in the data lake increases, our approach maintains more or less the same run time while

other methods slow down. As an example, let us consider the performance of “Parquet” on different table sizes. The average run time increases more than fourfold when we move from a 10 k table (Fig. 13(c)) to a 100 k table (Fig. 16(c)), going from 30 seconds to 142 seconds. On the other hand, our approach only shows a slight increase in the run time, from 38 seconds to 45 seconds.

Data Format: It is important to note that our approach is not intended to compete with other data formats. Rather, it is an optimization that can be applied to any of them. Our primary goal is to enhance the query performance

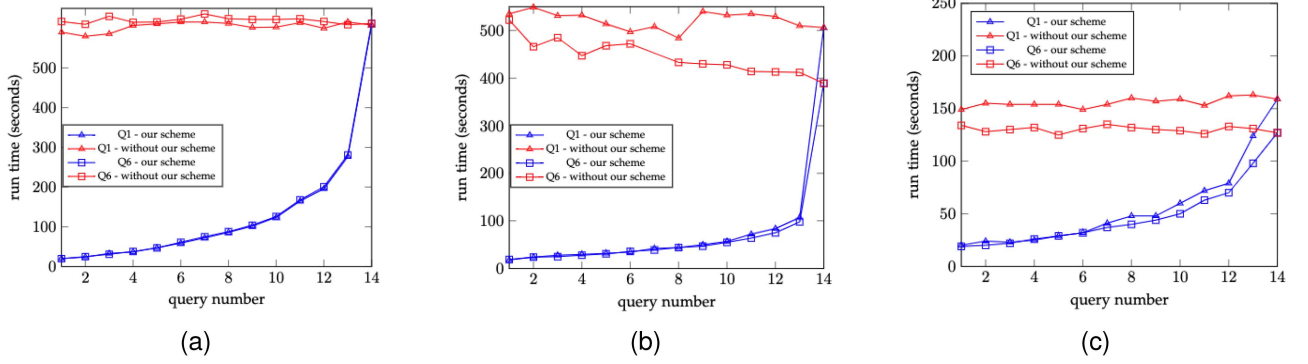


Fig. 16. Evaluation of TPC-H Q1 and Q6 on a table with 100,000 files. (a) CSV. (b) Iceberg. (c) Parquet.

TABLE VIII
RUN TIME DEGRADATION METRICS

#	format	$ F $	Fig.	query #	$ BQCPP_E $	$ BQCPP $	$ index $
1	csv	10,000	13a	12	1689	9515	25
2				13	2537	9900	29
3				6	386	4029	17
4	parquet	10,000	13c	7	513	5006	19
5				8	578	6000	19
6				9	731	7023	21
7				10	781	8013	21
8				11	1121	9032	23
9				12	1592	9515	25
10				13	2265	9900	29
11	parquet	20,000	14c	8	887	7425	18
12				9	1162	9002	19
13				10	1721	11042	22
14				11	2140	13812	23
15				12	2936	15364	25
16	parquet	50,000	15c	13	4528	18180	29
17				13	13007	30555	30

of a particular data lake by making minimal infrastructure changes.

By analyzing the evaluation results, we can clearly see the difference between the approaches. CSV is the slowest format, while Iceberg performs much faster than CSV but slower than Parquet. Parquet is by far the fastest of the three. The slowness of CSV is expected because it is the most basic scheme, and all files are read entirely for any execution. Iceberg and Parquet store data in columnar format and can skip irrelevant columns and rows when possible. In addition, Iceberg keeps metadata that is supposed to add additional functionality and improve query performance. Surprisingly, in our benchmark, we see only the overhead introduced by Iceberg, compared to Parquet, but no improvement. The reason may be that it needs careful tuning to work best while we use the default configuration.

As a result, our scheme outperformed Iceberg and CSV in virtually all experiments. The only exceptions were queries 12 and 13 in Fig. 13(a). With regard to “Parquet,” it partially outperformed us in tables with a low number of files (Figs. 13(c) and 14(c)). However, as the number of files grows, we perform much better. This is illustrated in Figs. 15(c) and 16(c).

2) *Analysis of Query Run Time Degradation:* As demonstrated in the previous section, our approach outperforms the

baseline in most experiments. Even in the small number of experiments where we run slower than the baseline, it is not a problem as long as we can anticipate it and revert to the default flow. Now, let us focus on the relevant metrics to gain insight into why, in some cases, we failed to detect the issue and what steps we can take to overcome it.

In Table VIII, we have all the relevant information regarding the queries that degraded baseline performance. For each query, the table presents (by the order of the table columns):

- serial number of the entry
- data format
- number of files in the data lake ($|F|$)
- figure containing this query
- query number
- estimated balanced coverage size ($|BQCPP_E|$)
- actual balanced coverage size ($|BQCPP|$)
- number of accessed index files by the balanced coverage plan ($|index|$)

To gain a better understanding of performance degradation, let us take a closer look at a specific query, entry #1 in Table VIII. For this query our scheme estimated the result size of the balanced coverage size to be 1689 files and the number of indexes to scan in the plan was 25. Thus, the total estimated cost of the balanced

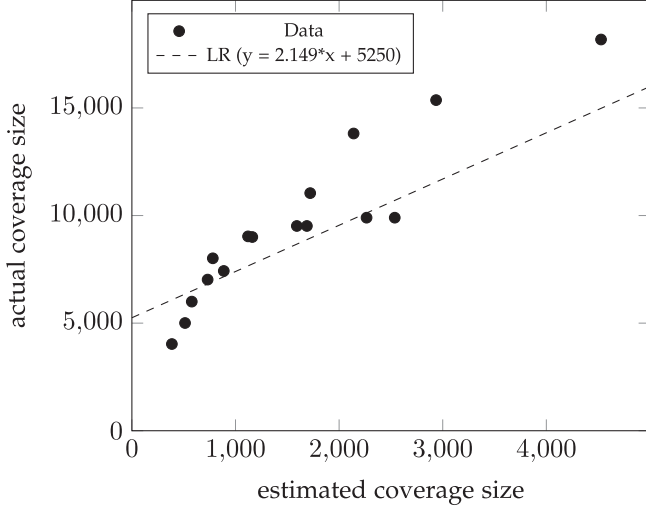


Fig. 17. Linear regression (estimated coverage size versus actual).

coverage plan was:

$$1689 + 25 = 1714 < K = |F| = 10,000 \quad (10)$$

As the estimated cost of the balanced coverage plan was lower than the set threshold K (which we set to be equal to the value of $|F|$ in all of our experiments), we proceeded to run the coverage plan as outlined in step 3 of Fig. 7. The actual balanced coverage plan turned out to be 9515, and the total query execution time was slower (as shown in query 13 of Fig. 13(a)). Thus, we encountered two issues during this experiment. First, our initial estimation was significantly off the mark (1689 vs. 9515), and second, even if we had predicted the actual result accurately, we would still have run our scheme and degraded performance (since $9515 < K = 10,000$).

To summarize, we have identified two problems in our approach: inaccurate estimation of a balanced coverage size and inappropriate value of threshold K . The first problem is not surprising since our estimation technique (Section IV-D) relies on simple statistical methods. To improve its accuracy, we can use the linear regression method. Fig. 17 presents a simple approach to regressing $|BQCPP_E|$ values from Table VIII onto $|BQCPP|$, and the numeric results of applying this approach are presented in Table IX (denoted by $|BQCPP'_E|$).

The second problem is also expected as setting threshold K to $|F|$ assumes that our approach does not incur any additional overhead besides reading files from the cloud storage. However, that is not the case; one example is that we need to intersect index files after reading them, which is a costly operation. Moreover, for different data formats, the threshold should be defined differently since, for example, the read operation for CSV format is much heavier than for Parquet or Iceberg, where only a subset of the file is actually read. Similarly, the number of data lake files also affects the threshold.

Choosing the optimal value of K in advance is not easy, but once we have the evaluation results, we can set it properly. Table IX shows the value of K (denoted as K') that should be defined in each case to ensure we fallback to the default flow

TABLE IX
ADJUSTED ESTIMATION AND K THRESHOLD

#	K'	$ BQCPP'_E $
1	$0.85 F = 8,500$	8879
2		10702
3	$0.6 F = 6,000$	6079
4		6352
5		6492
6		6820
7		6928
8		7659
9		8671
10		10117
11	$0.35 F = 7,000$	7156
12		7747
13		8948
14		9848
15		11559
16		14980
17	$0.5 F = 25,000$	33202

when needed. We have also verified that setting these values does not negatively affect other queries (those not presented in the table).

- 3) *Discussion:* Our experiments yield two main conclusions:
- 1) Our approach outperforms other schemes in most of cases, and two independent factors that make our approach faster are a low coverage degree and a high number of data lake files.
 - 2) Inaccurate estimation of results and inappropriate threshold choice may lead to our suggestion of using a coverage plan that results in worse total query time compared to the existing approach.

Conclusion (1) demonstrates that our approach is highly relevant, especially since there are many practical scenarios where users run queries with low coverage degree, as previously mentioned in Section II. In addition, real-world data lakes can contain billions of files [22], and our experiments have shown that even in data lakes with as few as 100,000 files, our scheme outperforms all other tested schemes for all queries.

Conclusion (2) is an important observation. It emphasizes the need to tune our scheme for a specific data lake to avoid query degradation. It can be done based on real user queries. While this approach is acceptable for real production systems, it would be more robust if we could eliminate this step. To achieve this, we could consider a potential solution based on the following approach: instead of assuming that we cannot access cloud storage during the estimation phase, we could “guess” that for some subsets of clauses, it is worth accessing the cloud for better estimations, which would provide us with a more accurate solution. In our future work, we plan to train machine learning models to assist us in making these “guesses”.

VI. CONCLUSION AND FUTURE DIRECTIONS

The amount of data being generated is constantly increasing, which has led to the development of new technologies to help manage this growth. One such solution is cloud data lakes, which have proven to be highly successful in dealing with Big Data

challenges. They offer excellent scalability, usability, and cost-effectiveness, but they are not very efficient when it comes to handling queries.

In this paper, we analyze the problem of poor query performance in cloud data lakes. We identify the main obstacles to achieving better performance and formally define the problem by introducing a new concept of the query (tight) coverage set. We use our terminology to define an optimization problem (BQCPP) that finds the best (balanced) coverage set for the given query. We show that BQCPP is NP-hard in its most precise version (when the result is estimated by files or records). We deal with the hardness of BQCPP by two strategies: suggest heuristic algorithms and estimate the result by the number of records (rather than actual files or records). Our solution is based on ideas from relational databases related to indexing and statistics management. The main idea is that the storage resources are usually much cheaper than the compute resources [12], [44], so if we could provably improve query performance by adding more storage, it probably would be very useful for many data lake users. We demonstrate through the experiments that our approach works well and outperforms the existing approaches in most cases.

Our solution focuses on a very specific niche in a query optimization area - reducing the number of files read from the storage. That is why our cost model is solely based on reads from the cloud. There are many other possible optimizations, most known of which were presented in Section III, that are orthogonal to our work and can be easily combined with. Moreover, our scheme itself can be used for additional optimizations. For example, our index data structure can be trivially used for the optimization of index-only accesses [49], like finding min, max, count, sum, average, and distinct values. The coverage set of a join query can be computed by joining corresponding index files. An interesting caching technique can be based on the tight coverage sets: $TCov$ function defines an equivalence relation on a set of all possible queries over the table, and we can say that two queries Q_1 and Q_2 are in the same equivalence class if and only if there is X such that $TCov(X, Q_1)$ and $TCov(X, Q_2)$. The intuition behind this statement is that many queries have the same tight coverage set (i.e., need to read the same files from the data lake). So, if for any query Q we could tell for each equivalence class it belongs, we would be able to cache tight coverage set per equivalence class on the query engine side and, by that, improve query performance even more while using limited memory resources. Another cache-related approach that is also worth considering is the caching of index parts on the user side. Previous work concentrated on caching on the query engine side, as the assumption was that a parallel execution engine should process each query anyway. But once we start using indexing, we do not have to use a cluster of nodes for each query anymore, and a single user's machine can perform query processing end-to-end for many types of queries. Then, caching techniques can be focused on storing "hot" index sections on the user's machine (disk or memory). We are planning to study all these potential optimizations in our future work.

ACKNOWLEDGMENTS

The authors would like to thank anonymous reviewers for providing them with valuable feedback. The problem presented in this paper and the outline of the solution were published in [50].

REFERENCES

- [1] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The google file system," in *Proc. 19th ACM Symp. Operating Syst. Princ.*, 2003, pp. 29–43.
- [2] J. D. Fay Chang et al., "Bigtable: A distributed storage system for structured data," *ACM Trans. Comput. Syst.*, vol. 26, no. 4, pp. 1–4, 2006.
- [3] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Syst. Technol.*, 2010, pp. 1–10.
- [4] A. Gupta et al., "Amazon redshift and the case for simpler data warehouses," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 1917–1923.
- [5] Amazon, AWS S3, 2023. [Online]. Available: <https://aws.amazon.com/s3>
- [6] Google, Google cloud storage, 2023. [Online]. Available: <https://cloud.google.com/storage>
- [7] Microsoft, Azure blob storage, 2023. [Online]. Available: <https://azure.microsoft.com/en-us/services/storage/blobs>
- [8] M. Mesnier, G. R. Ganger, and E. Riedel, "Object-based storage," *IEEE Commun. Mag.*, vol. 41, no. 8, pp. 84–90, Aug. 2003.
- [9] M. Armbrust et al., "Delta lake: High-performance acid table storage over cloud object stores," in *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.
- [10] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," in *Proc. Annu. Conf. Innov. Data Syst. Res.*, 2021. [Online]. Available: https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- [11] Y. Yang et al., "FlexPushdownDB: Hybrid pushdown and caching in a cloud DBMS," in *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2101–2113, 2021.
- [12] J. Tan et al., "Choosing a cloud DBMS: Architectures and tradeoffs," in *Proc. VLDB Endowment*, vol. 12, no. 12, pp. 2170–2182, 2019.
- [13] D. Abadi et al., "The seattle report on database research," *Commun. ACM*, vol. 65, no. 8, pp. 72–79, 2022.
- [14] M. Zaharia et al., "Apache spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [15] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [16] Apache, Apache parquet, 2023. [Online]. Available: <https://parquet.apache.org/>
- [17] Apache, Apache orc, 2023. [Online]. Available: <https://orc.apache.org/>
- [18] J. Camacho-Rodríguez et al., "Apache hive: From MapReduce to enterprise-grade Big Data warehousing," in *Proc. Int. Conf. Manage. Data*, 2019, pp. 1773–1786.
- [19] M. Armbrust et al., "Spark sql: Relational data processing in spark," in *Proc. ACM SIGMOD Int. Conf. Manage. data*, 2015, pp. 1383–1394.
- [20] R. Sethi et al., "Presto: SQL on everything," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 1802–1813.
- [21] M. Armbrust et al., "A view of cloud computing," *Commun. ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [22] P. Jain, P. Kraft, C. Power, T. Das, I. Stoica, and M. Zaharia, "Analyzing and comparing lakehouse storage systems," in *Proc. Conf. Innov. Data Syst. Res.*, 2023. [Online]. Available: <https://www.cidrdb.org/cidr2023/papers/p92-jain.pdf>
- [23] I. G. P. Team, *EU General Data Protection Regulation (GDPR)—An implementation and compliance guide*, Ely, U.K.: IT Governance Ltd., 2020.
- [24] E. F. Codd, "Extending the database relational model to capture more meaning," *ACM Trans. Database Syst.*, vol. 4, no. 4, pp. 397–434, 1979.
- [25] R. Ramakrishnan, J. Gehrke, and J. Gehrke, *Database Management Systems*, vol. 3. New York, NY, USA: McGraw-Hill, 2003.
- [26] K. Miettinen, *Nonlinear Multiobjective Optimization*, vol. 12. Berlin, Germany: Springer, 1999.
- [27] M. Saxena et al., "The story of aws glue," in *Proc. VLDB Endow.*, vol. 16, no. 12, pp. 3557–3569, 2023.
- [28] P. Edara and M. Pasumansky, "Big metadata: When metadata is Big Data," in *Proc. VLDB Endowment*, vol. 14, no. 12, pp. 3083–3095, 2021.

- [29] G. Moerkotte, “Small materialized aggregates: A light weight index structure for data warehousing,” in *Proc. 24rd Int. Conf. Very Large Data Bases*, A. Gupta, O. Shmueli, and J. Widom, Eds, New York City, NY, USA, Morgan Kaufmann, Aug. 24–27, 1998, pp. 476–487. [Online]. Available: <http://www.vldb.org/conf/1998/p476.pdf>
- [30] C. Liu, A. Pavlenko, M. Interlandi, and B. Haynes, “A deep dive into common open formats for analytical dbms,” in *Proc. VLDB Endowment*, vol. 16, no. 11, pp. 3044–3056, 2023.
- [31] P. Ta-Shma, G. Khazma, G. Lushi, and O. Feder, “Extensible data skipping,” in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 372–382.
- [32] L. Sun, M. J. Franklin, S. Krishnan, and R. S. Xin, “Fine-grained partitioning for aggressive data skipping,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1115–1126.
- [33] L. Sun, M. J. Franklin, J. Wang, and E. Wu, “Skipping-oriented partitioning for columnar layouts,” in *Proc. VLDB Endowment*, vol. 10, no. 4, pp. 421–432, 2016.
- [34] Z. Yang et al., “QD-tree: Learning data layouts for Big Data analytics,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 193–208.
- [35] Microsoft, Data lake storage query acceleration, 2023. [Online]. Available: <https://learn.microsoft.com/en-gb/azure/storage/blobs/data-lake-storage-query-acceleration>
- [36] AWS, Amazon S3 select, 2023. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/selecting-content-from-objects.html>
- [37] B. Dageville et al., “The snowflake elastic data warehouse,” in *Proc. Int. Conf. Manage. Data*, 2016, pp. 215–226.
- [38] Google, Google bigquery, 2023. [Online]. Available: <https://cloud.google.com/bigquery>
- [39] A. Behm et al., “Photon: A fast query engine for lakehouse systems,” in *Proc. Int. Conf. Manage. Data*, 2022, pp. 2326–2339.
- [40] S. Vakharina, P. Li, W. Liu, and S. Narayanan, “Shared foundations: Modernizing meta’s data lakehouse,” in *Proc. Conf. Innov. Data Syst. Res.*, 2023. [Online]. Available: <https://www.cidrdb.org/cidr2023/papers/p77-chattopadhyay.pdf>
- [41] Apache, Apache iceberg, 2023. [Online]. Available: <https://iceberg.apache.org/>
- [42] Apache, Apache hudi, 2023. [Online]. Available: <https://hudi.apache.org/>
- [43] Amazon, Building and maintaining an amazon S3 metadata index without servers, 2015. [Online]. Available: <https://tinyurl.com/3m7wa35f>
- [44] G. Weintraub, E. Gudes, and S. Dolev, “Needle in a haystack queries in cloud data lakes,” in *Proc. Int. Conf. Extending Database Technol./Database Theory Workshops*, 2021, Art. no. 125.
- [45] A. Brodnik and J. I. Munro, “Membership in constant time and almost-minimum space,” *SIAM J. Comput.*, vol. 28, no. 5, pp. 1627–1640, 1999.
- [46] G. Weintraub, Data-lake-coverage, 2023. [Online]. Available: <https://github.com/grishaw/data-lake-coverage>
- [47] A. Silberschatz et al., *Database System Concepts*, vol. 5. New York, NY, USA: McGraw-Hill, 2002.
- [48] TPC, TPC-H, 2023. [Online]. Available: <https://www.tpc.org/tpch/>
- [49] C. Mohan, D. Haderle, Y. Wang, and J. Cheng, “Single table access using multiple indexes: Optimization, execution, and concurrency control techniques,” in *Proc. Int. Conf. Extending Database Technol.*, Springer, 1990, pp. 29–43.
- [50] G. Weintraub, “Optimizing cloud data lakes queries,” in *Proc. Conf. Very Large Data Bases*, 2023, pp. 13–16.



Grisha Weintraub received the BSc and MSc degrees from the Open University of Israel. Currently, he is working towards the PhD degree with the Department of Computer Science, Ben-Gurion University. His research focuses on developing innovative solutions to tackle (Big) Data management challenges. In addition to pursuing an academic path, he is an experienced industry professional, currently working as a Big Data Team Lead with IBM.



Ehud Gudes received the BSc and MSc degrees from the Technion - Israel Institute of Technology, and the PhD degree in computer and information science from Ohio State University in 1976. Following his PhD, he worked both in academia (Penn State University, Ben-Gurion University), where he did research in the areas of Database Systems and Data Security, and in Industry (Wang Labs, National Semi, IBM), where he developed Query languages, CAD Software, and Expert systems. He has published more than 120 papers in the above general areas, and was the chair of several international conferences, including the 2002 and 2009 IFIP WG11.3 conference on Data and Application security and the IFIPTM 2014 conference on Trust management. He is currently a professor with Ben-Gurion university and leading several research projects in data security. His research interests encompass the domain of knowledge and databases, data security including trust and privacy, and data mining especially privacy-preserving data mining.



Shlomi Dolev (Fellow, IEEE) received the DSc degree in computer science from the Technion in 1992. He is the founder and the first department head of the Computer Science Department at Ben-Gurion University, established in 2000. He is the author of a book entitled “Self-Stabilization” published by MIT Press in 2000. His publications include more than three hundred publications. He is EAI fellow, and served in more than a hundred program committees, chairing several including the two leading conferences in distributed computing, DISC 2006, and PODC 2014. He holds the Ben-Gurion university Rita Altura trust chair in Computer Sciences. From 2011 to 2014, he served as the dean of the Natural Sciences Faculty with Ben-Gurion University of the Negev. From 2010 to 2016, he has served as head of the Inter University Computation Center of Israel. He currently serves as the steering committee head of the Computer Science discipline of the Israeli ministry of education.



Jeffrey D. Ullman received the BS degree from Columbia University in 1963 and the PhD degree from Princeton in 1966. He is the Stanford W. Ascherman professor of engineering (Emeritus) with the Department of Computer Science at Stanford and CEO of Gradience Corp. Prior to his appointment at Stanford in 1979, he was a member of the technical staff of Bell Laboratories from 1966 to 1969, and on the faculty of Princeton University between 1969 and 1979. From 1990 to 1994, he was chair of the Stanford Computer Science Department. He has served as the chair of the CS-GRE Examination board, a member of the ACM Council, the chair of the New York State CS Doctoral Evaluation Board, on several US National Science Foundation advisory boards, and is past or present editor of several journals. Ullman was elected to the National Academy of Engineering in 1989 and has held Guggenheim and Einstein Fellowships. He has received the Sigmod Contributions Award in 1996, the ACM Karl V. Karlstrom Outstanding Educator Award in 1998, the Knuth Prize in 2000, the Sigmod E. F. Codd Innovations award in 2006, and the IEEE von Neumann medal in 2010. He is the author of 16 books, including widely read books on database systems, compilers, automata theory, and algorithms.